



LLD – DAY 2

Concept of OOPS

Pillars Covered

1. **Abstraction**
 2. **Encapsulation**
-

◇ Why Do We Need OOPS?

✗ Problems Without OOPS (Procedural Programming)

- Data openly accessible
 - No data security
 - Code duplication
 - Difficult to maintain large systems
 - Hard to scale applications
-

☑ Advantages of OOPS

- Real-world modeling
 - Data security
 - Better code organization
 - Reusability
 - Maintainability
 - Scalability (important for LLD)
-

🧠 Real-World Example

Car System

- You can press accelerator
- You cannot directly change engine logic
- You cannot set speed arbitrarily

👉 This real-world behavior is achieved using **OOPS**.

◇ OOPS in LLD Context

- OOPS helps design **classes & objects**
- LLD uses OOPS to define:
 - Responsibilities

- Boundaries
 - Interactions
- OOPS is the **foundation of LLD**
-

◇ 1. Abstraction

☑ Definition

Abstraction means showing **what an object can do** and hiding **how it does it**.

✎ Focus is on **behavior**, not implementation.

🔑 Key Points of Abstraction

- Hides internal complexity
 - Exposes only essential operations
 - Reduces coupling
 - Improves flexibility
 - Helps in interface-based design
-

🧠 How Abstraction is Achieved in C++

- Abstract classes
 - Pure virtual functions
-

🔗 Real-World Examples of Abstraction

- Car pedals
- ATM machine
- TV remote
- Mobile phone UI

User knows **WHAT** action to perform, not **HOW** it works internally.

💡 Abstraction in LLD

- Used to define **interfaces**
 - Used to separate **contract and implementation**
 - Helps in swapping implementations without affecting client code
-

✎ Abstraction Code Example (Concept Summary)

- Abstract class defines **WHAT**
- Child class defines **HOW**
- Object is accessed using **parent reference**

◇ 2. Encapsulation

☑ Definition

Encapsulation is the process of binding **data and methods together** and restricting direct access to data.

🔒 Important Point

Data Security is the MOST important benefit of Encapsulation

🧠 How Encapsulation is Achieved

1. Make class variables **private**
2. Provide access using **public methods**

✗ Without Encapsulation

```
car.speed = 500;    // Invalid & dangerous
```

☑ With Encapsulation

```
car.getSpeed();  
car.setSpeed(120);
```

Access is **controlled and validated**.

🔗 Real-World Example of Encapsulation

- ATM balance (cannot directly modify)
- Bank account details
- Mobile internal hardware

💡 Encapsulation in LLD

- Protects object state
- Prevents misuse of data
- Makes system stable & secure
- Helps in maintaining invariants

◇ Difference Between Abstraction and Encapsulation

★ VERY IMPORTANT

Feature	Abstraction	Encapsulation
Meaning	Hide implementation	Hide data
Focus	WHAT	HOW
Purpose	Reduce complexity	Data security
Achieved using	Abstract class / Interface	Access modifiers
Example	Car pedals	Private speed variable
LLD Role	Defines contracts	Protects state

◇ Relation Between Abstraction & Encapsulation

- Abstraction decides **what to expose**
- Encapsulation decides **how to protect it**
- Both work together for clean design

Day 2 Summary (Revision Sheet)

✓ OOPS needed for real-world systems ✓ Abstraction hides complexity ✓ Encapsulation hides data ✓
Data security comes from encapsulation ✓ Both are core pillars of LLD

Exam / Interview Golden Lines

- “Abstraction hides implementation, Encapsulation hides data.”
- “Encapsulation provides data security.”
- “LLD is built on top of OOPS principles.”